CodeAlpha — Cyber Security Internship

TASK 4: Network Intrusion Detection System (NIDS)

Using Snort

Intern Name: Musfira Hassan

Student ID: CA/DF1/201508

Domain: Cyber Security

Tool Used: Snort (Network-based Intrusion Detection System)

Environment: Ubuntu Snort IDS (192.168.19.136) | Windows Target (192.168.134.173) | Attacker Kali/Parrot (192.168.19.134)

Lab IP Summary — Attacker (Kali/Parrot): 192.168.19.134 | Windows Target: 192.168.134.173 | Ubuntu Snort/System: 192.168.19.136

1. Objective

The goal of this task is to set up and configure a Network-based Intrusion Detection System (NIDS) using Snort. Custom detection rules are written to identify suspicious and malicious network activity, including ICMP, HTTP/HTTPS, TCP, and UDP traffic. Attack traffic is simulated from a Linux-based system, and Snort running on Ubuntu monitors the network, generates alerts, and logs captured packets for analysis.

2. CodeAlpha Task Requirements Covered

- Set up a network-based IDS using Snort
- Configure custom rules and alerts to detect suspicious/malicious activity
- Monitor network traffic for potential threats (ICMP, HTTP/HTTPS, TCP, UDP)
- Capture and log matching packets for analysis
- Demonstrate response/alerting when rules are triggered

3. Lab Setup & Architecture

A multi-machine lab environment was used so Snort could observe real attack and application traffic:

Role summary from screenshots — Attacker: 192.168.19.134 | Windows Target: 192.168.134.173 | Ubuntu (Snort / monitored system): 192.168.19.136.

- **Sensor:** Ubuntu machine — Snort IDS installed and running (sensor / monitor)
- **Target:** Windows machine — target system whose traffic is monitored
- **Attacker:** Parrot OS / Linux — attacker machine used to generate flood and probe traffic

Target Machine IP Address: 192.168.134.173 (Windows)

Ubuntu (Snort IDS) IP Address: 192.168.19.136

Attacker Machine IP Address: 192.168.19.134 (Kali / Parrot Linux)

Own System IP (for TCP capture rules): 192.168.19.136 (Ubuntu)

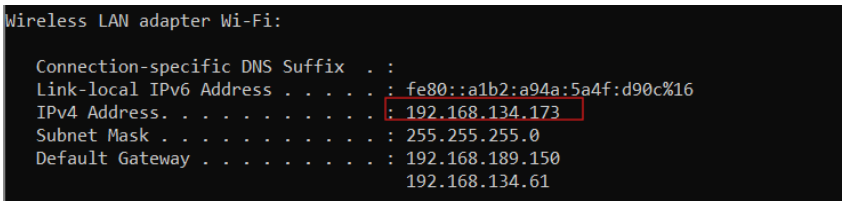
IP addresses below were taken from the lab screenshots (ipconfig / ip addr / Snort alerts).

4. Part A — Protocol Attack Detection (ICMP, TCP, HTTP, UDP)

Custom Snort rules were created to detect ICMP, TCP, HTTP, and UDP traffic/attacks. Flood and probe attacks were simulated from the attacker machine (192.168.19.134) toward the Windows target (192.168.134.173). Snort running on Ubuntu (192.168.19.136) monitored the traffic and triggered alerts when rule conditions matched.

TARGET MACHINE IP ADDRESS: 192.168.134.173

Figure: Windows ipconfig output. The highlighted IPv4 address of the target machine is 192.168.134.173 (Wireless LAN adapter Wi-Fi).



4.1 ICMP Rule & Attack

Description: A custom Snort rule detects ICMP ping traffic toward the home/target network and alerts with "ICMP Ping Detected" (sid:1000001).

Figure: Custom ICMP Snort rule used for ping detection (sid:1000001).

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
```

ATTACK — ICMP Ping / hping3 from Attacker

Description: From the attacker machine (192.168.19.134), ICMP traffic was sent to the Windows target (192.168.134.173) using ping and hping3 -1. Snort on Ubuntu raised "ICMP Ping Detected" alerts showing 192.168.19.134 → 192.168.134.173.

Figure (Evidence): Attacker terminal pinging 192.168.134.173 while Ubuntu Snort logs "ICMP Ping Detected" (192.168.19.134 → 192.168.134.173).

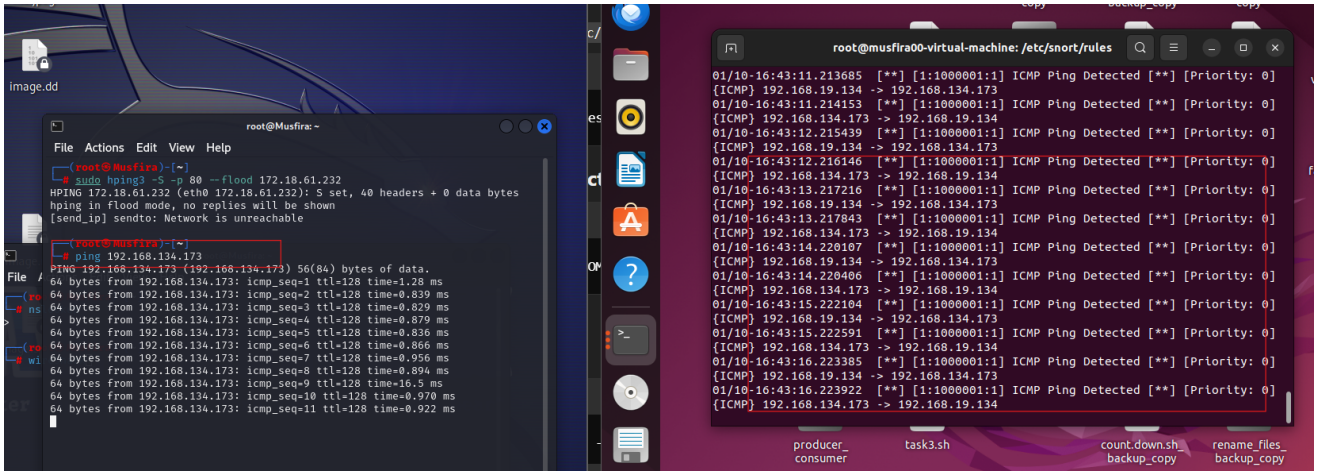
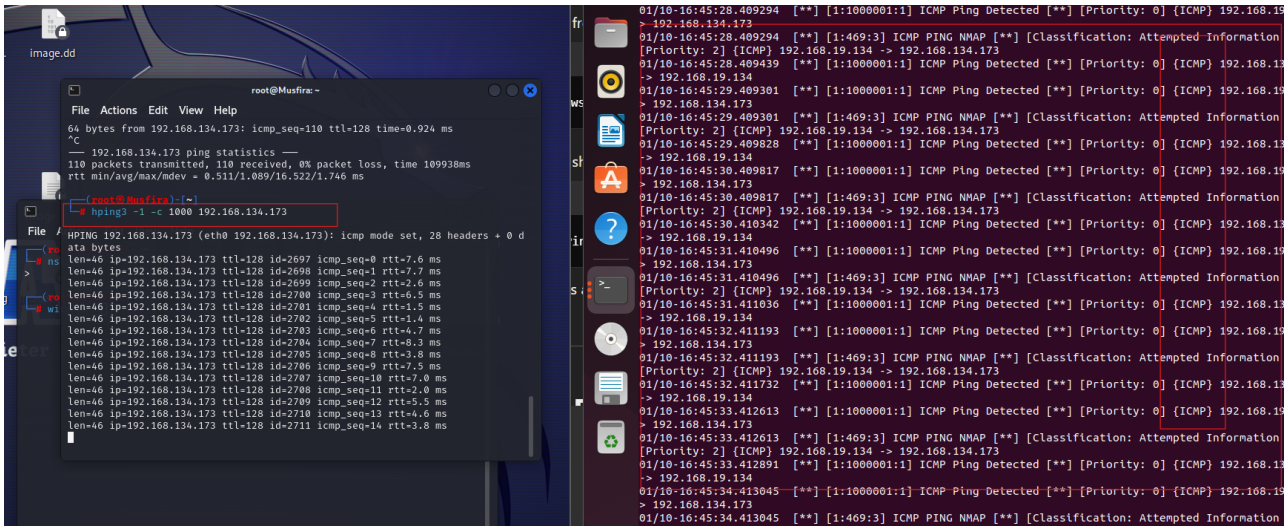


Figure (Evidence): hping3 -1 ICMP traffic from attacker toward 192.168.134.173; Snort continues alerting on ICMP from 192.168.19.134.



4.2 TCP Rule & Attack

Description: Snort rule to detect TCP SYN packets toward $HOME_NET port 80 (possible SYN flood). Message: “SYN Flood Detected” (sid:1000002).

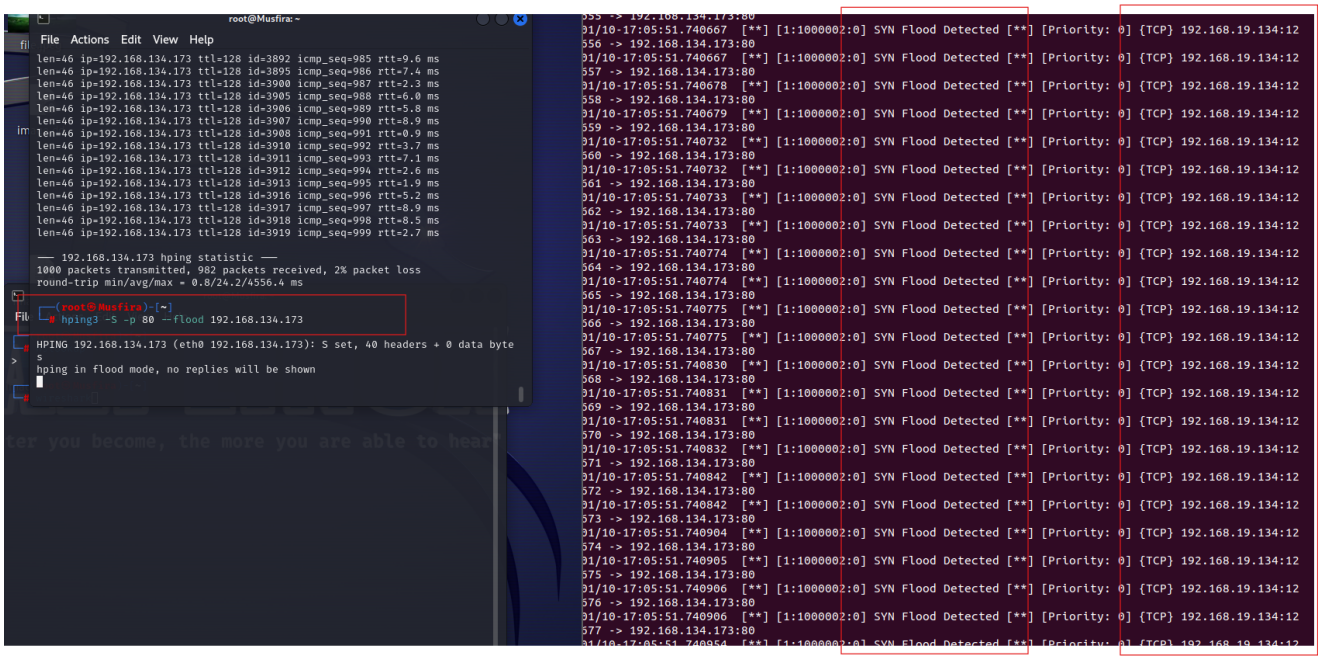
Figure: Custom TCP SYN-flood Snort rule (sid:1000002).

```
alert tcp any any -> $HOME_NET 80 (flags:S; msg:"SYN Flood Detected"; sid:1000002;)
```

ATTACK — TCP SYN Flood (hping3)

Description: A TCP SYN flood was launched from attacker 192.168.19.134 against target 192.168.134.173 on port 80 using hping3 -S -p 80 --flood. Snort triggered “SYN Flood Detected” alerts (sid:1000002) for 192.168.19.134 → 192.168.134.173:80.

Figure (Evidence): hping3 SYN flood against 192.168.134.173:80 detected as “SYN Flood Detected” (192.168.19.134 → 192.168.134.173:80).



4.3 HTTP Rule & Attack

Description: Snort rule to detect HTTP access on port 80 toward the protected network. Message: “HTTP Access Detected” (sid:1000003).

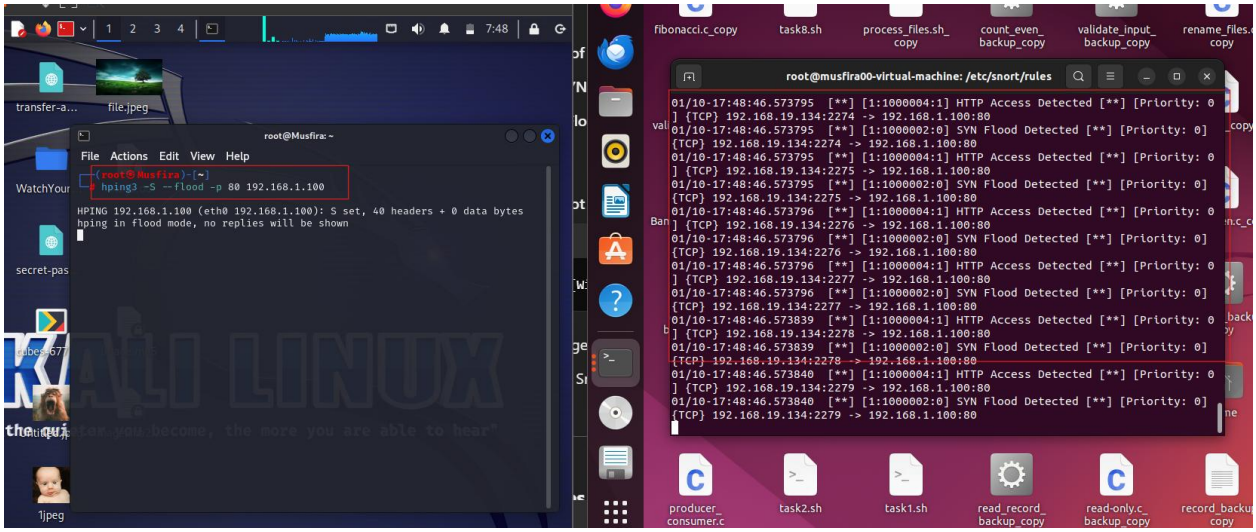
Figure: Custom HTTP-access Snort rule (sid:1000003).

```
alert tcp any any -> $HOME_NET 80 (msg:"HTTP Access Detected"; sid:1000003; rev:1;)
```

ATTACK — HTTP / Port-80 Flood Test

Description: HTTP-related attack traffic toward port 80 was generated with hping3. In this test screenshot the destination is 192.168.1.100:80. Snort logged “HTTP Access Detected” (and related SYN flood alerts) with source 192.168.19.134.

Figure (Evidence): Port-80 flood/HTTP test; Snort alerts show source 192.168.19.134 with “HTTP Access Detected” / “SYN Flood Detected”.



4.4 UDP Rule & Attack

Description: Snort rule to detect UDP flood traffic toward $HOME_NET. Message: “UDP Flood Detected” (sid:1000004).

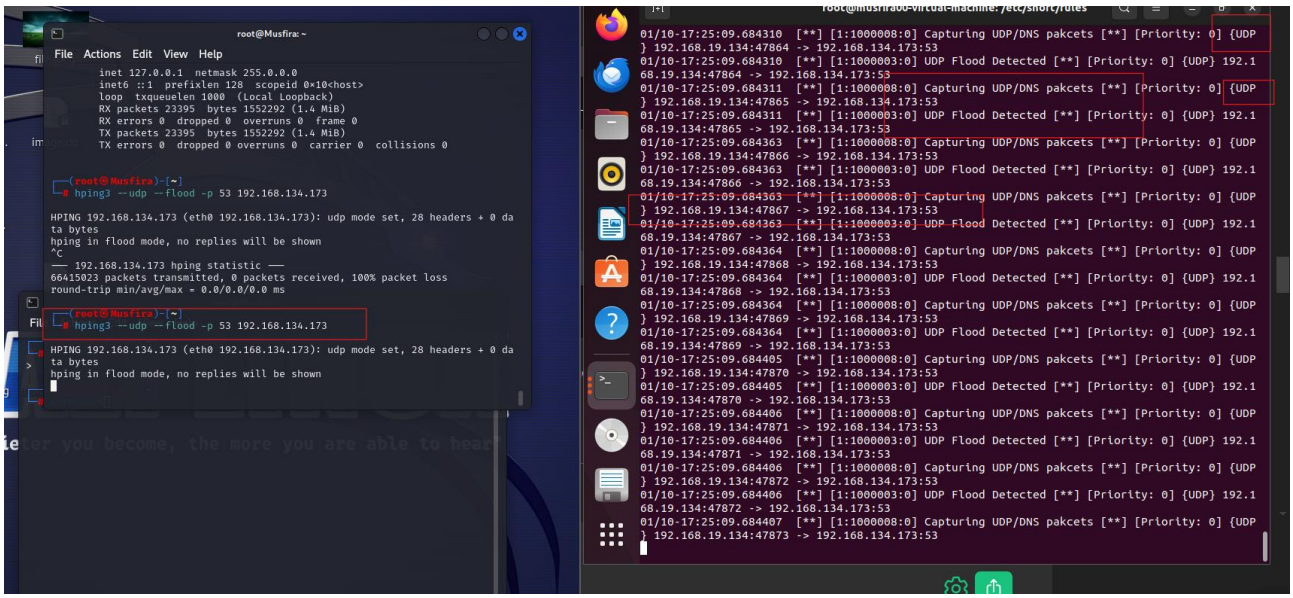
Figure: Custom UDP-flood Snort rule (sid:1000004).

```
alert udp any any -> $HOME_NET any (msg:"UDP Flood Detected"; sid:1000004; rev:1;)
```

ATTACK — UDP Flood to DNS Port 53

Description: A UDP flood was sent from attacker 192.168.19.134 to target 192.168.134.173 on port 53 using hping3 --udp --flood -p 53. Snort raised “UDP Flood Detected” alerts for 192.168.19.134 → 192.168.134.173:53.

Figure (Evidence): UDP flood to 192.168.134.173:53 with Snort alerts “UDP Flood Detected” (192.168.19.134 → 192.168.134.173:53).



5. Part B — HTTP/HTTPS Website Visit Detection

Snort rules were written to generate alerts when users visit specific websites over HTTP or HTTPS. This demonstrates content/destination-based monitoring of web traffic.

5.1 http://testphp.vulnweb.com/

IP ADDRESS: 44.228.249.3 (testphp.vulnweb.com)

Figure: nslookup / DNS resolution for testphp.vulnweb.com showing IP 44.228.249.3.

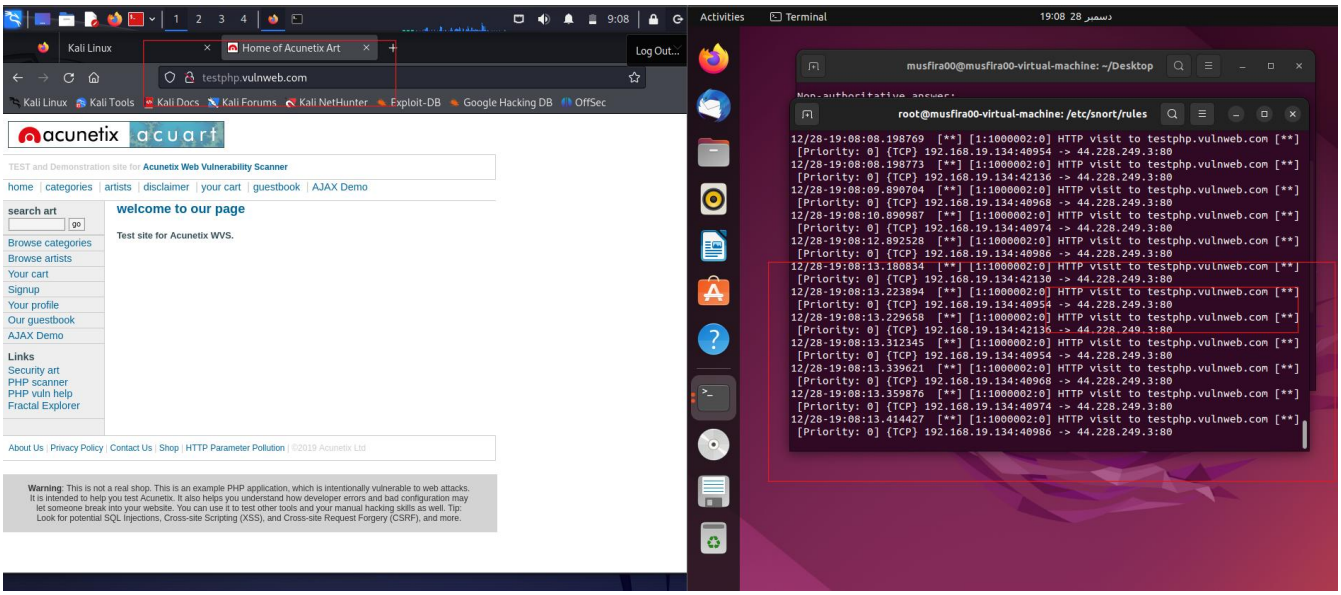
```
musfira00@musfira00-virtual-machine:~/Desktop$ nslookup
> testphp.vulnweb.com
Server:         127.0.0.53
Address:        127.0.0.53#53
```

RULE:

Figure: Snort rules alerting on HTTP/HTTPS visits to testphp.vulnweb.com (44.228.249.3) — ports 80 and 443.

```
alert tcp any any -> 44.228.249.3 80 (msg: "HTTP visit to testphp.vulnweb.com"; sid:1000002;)
alert tcp any any -> 44.228.249.3 443 (msg: "HTTPS visit to testphp.vulnweb.com"; sid:1000003;)
```

Figure (Evidence): Visiting testphp.vulnweb.com triggered Snort alert — source 192.168.19.134 → destination 44.228.249.3:80.



5.2 http://www.altoromutual.com/

IP ADDRESS: 65.61.137.117 (www.altoromutual.com)

Figure: nslookup / DNS resolution for www.altoromutual.com showing IP 65.61.137.117.

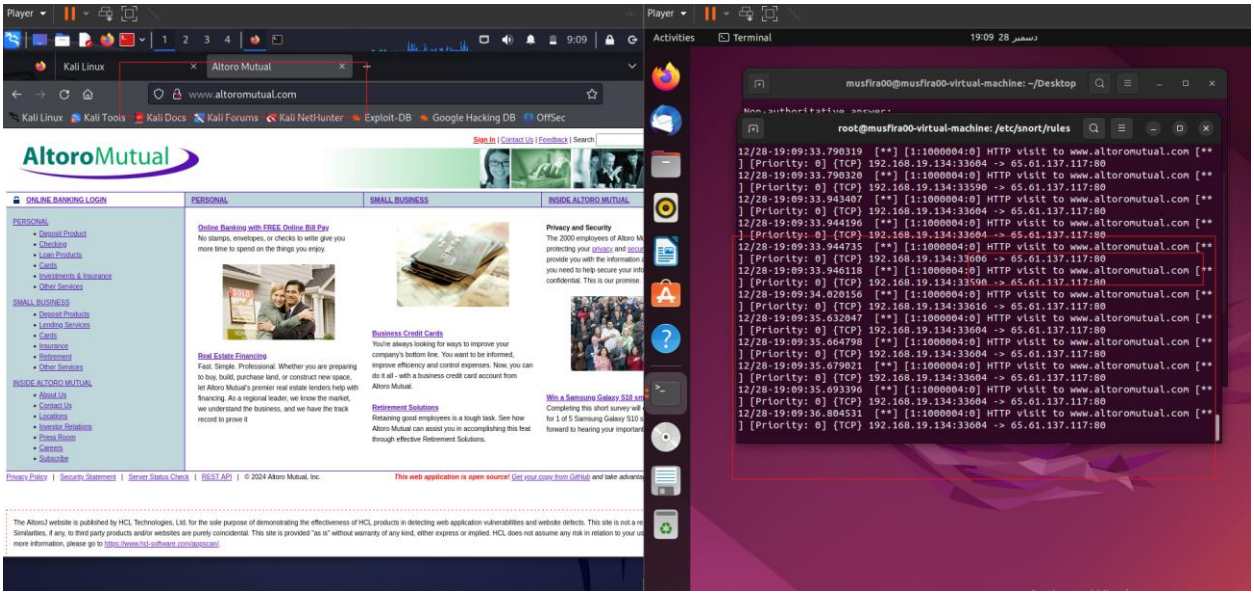
```
> www.altoromutual.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
www.altoromutual.com    canonical name = altoromutual.com.
Name:   altoromutual.com
Address: 65.61.137.117
> www.geeksforgeeks.org
Server:         127.0.0.53
Address:        127.0.0.53#53
```

RULE:

Figure: Snort rules alerting on HTTP/HTTPS visits to www.altoromutual.com (65.61.137.117) — ports 80 and 443.

```
alert tcp any any -> 65.61.137.117 80 (msg: "HTTP visit to www.altoromutual.com"; sid:1000004;)
alert tcp any any -> 65.61.137.117 443 (msg: "HTTPS visit to www.altoromutual.com"; sid:1000005;)
```

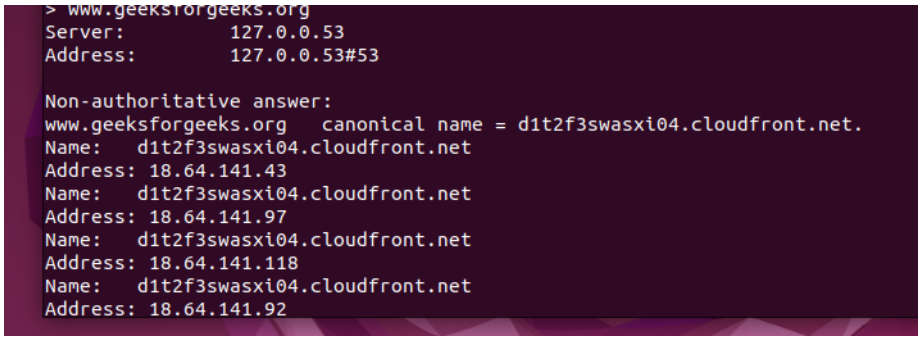
Figure (Evidence): Visiting www.altoromutual.com triggered HTTP/HTTPS visit rules for 65.61.137.117.



5.3 https://www.geeksforgeeks.org/

IP ADDRESS: 18.64.141.118 (www.geeksforgeeks.org)

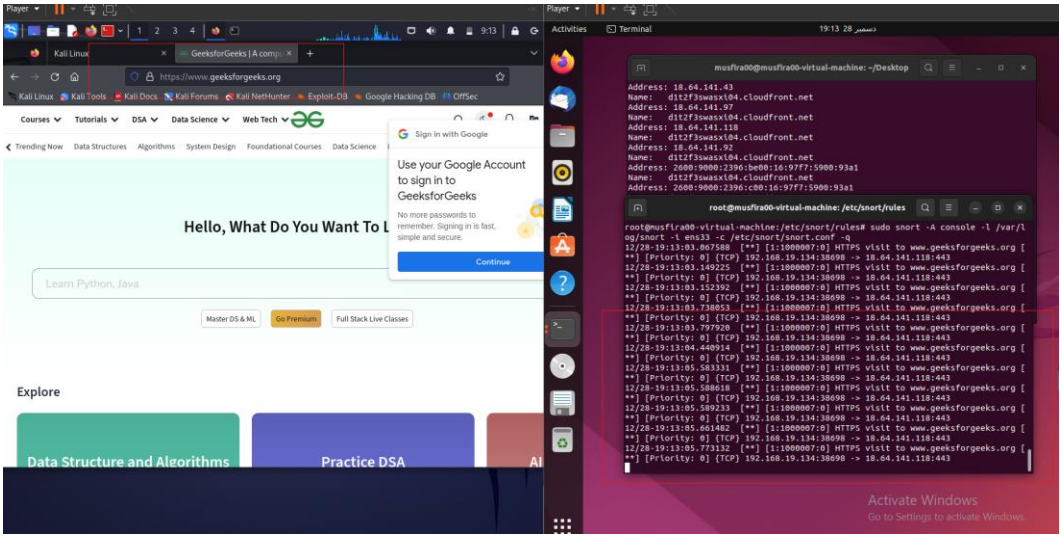
Figure: nslookup / DNS resolution for www.geeksforgeeks.org showing IP 18.64.141.118.



RULE:

Figure: Snort rules alerting on HTTP/HTTPS visits to www.geeksforgeeks.org (18.64.141.118) — ports 80 and 443.

Figure (Evidence): Visiting www.geeksforgeeks.org triggered HTTP/HTTPS visit rules for 18.64.141.118.



6. Part C — Detect UDP/DNS Packet Capture Attack

Description: A Snort rule detects potential UDP/DNS packet-capture / flood style attacks (e.g., hping3). It watches UDP traffic from attacker 192.168.19.134 to any destination on DNS port 53 (msg: “UDP/DNS packet capture attack”; sid:1000008).

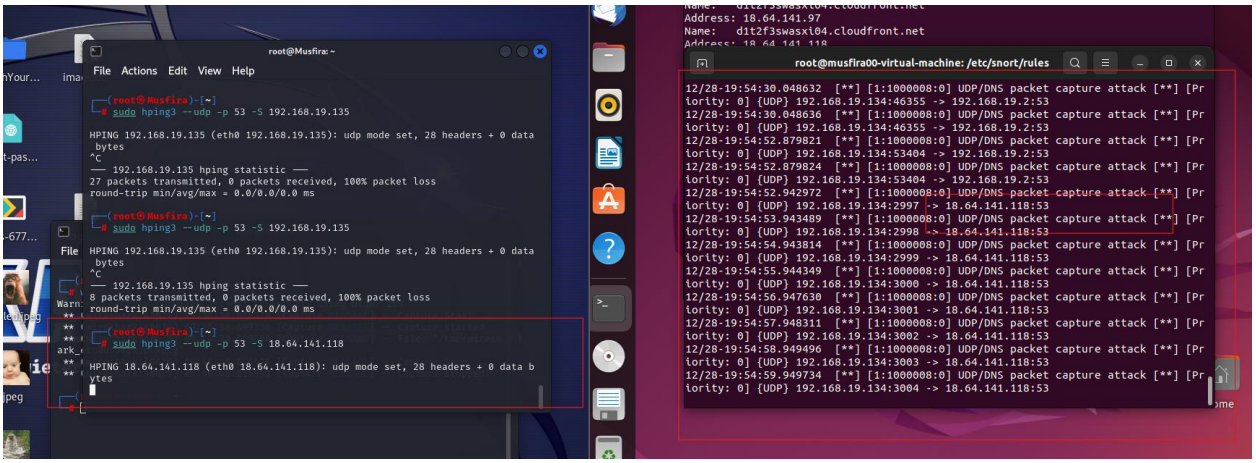
RULE:

Figure: Snort rule monitoring UDP/DNS attack traffic from attacker 192.168.19.134 to any host on port 53 (sid:1000008).

```
alert udp 192.168.19.134 any -> any 53 (msg: "UDP/DNS packet capture attack"; sid:1000008;)
```

SNORT CAPTURING PACKETS / ALERTS:

Figure (Evidence): Live Snort alerts while UDP/DNS attack traffic from 192.168.19.134 is captured and logged.



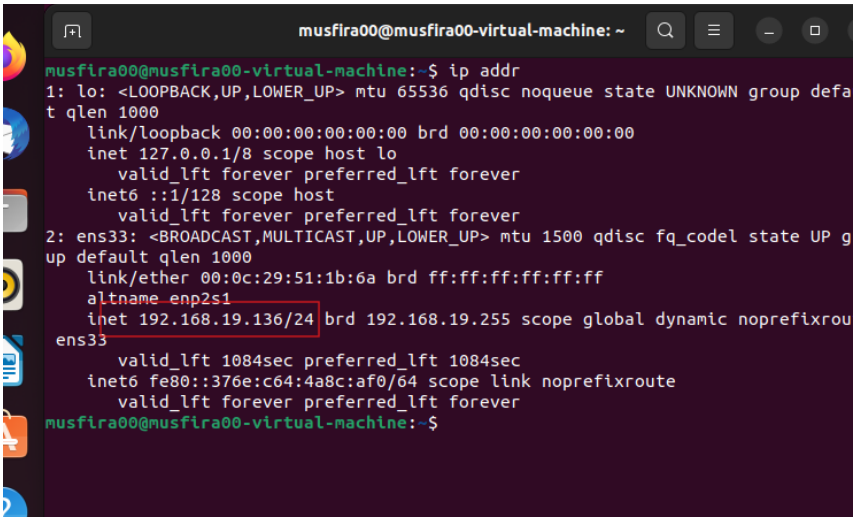
7. Part D — TCP Packet Capture for Specific IP

TCP Packet Capture for Specific IP

Description: Snort rules alert on TCP packets to/from the Ubuntu system IP 192.168.19.136 (from ip addr on musfira00-virtual-machine), covering incoming traffic to the host and outgoing TCP sessions.

System IP address: 192.168.19.136 (Ubuntu — ens33)

Figure: ip addr output on Ubuntu. Highlighted IPv4 address of the monitored system is 192.168.19.136.



First Rule — Incoming TCP to the system (192.168.19.136:80): alert tcp any any -> 192.168.19.136 80 (msg:"Incoming tcp Packets to this system"; sid:1000009;)

Figure: Incoming TCP capture/alert rule.

```
alert tcp any any -> 192.168.19.136 80 (msg: "Incoming tcp Packets to this system"; sid 1000009;)
```

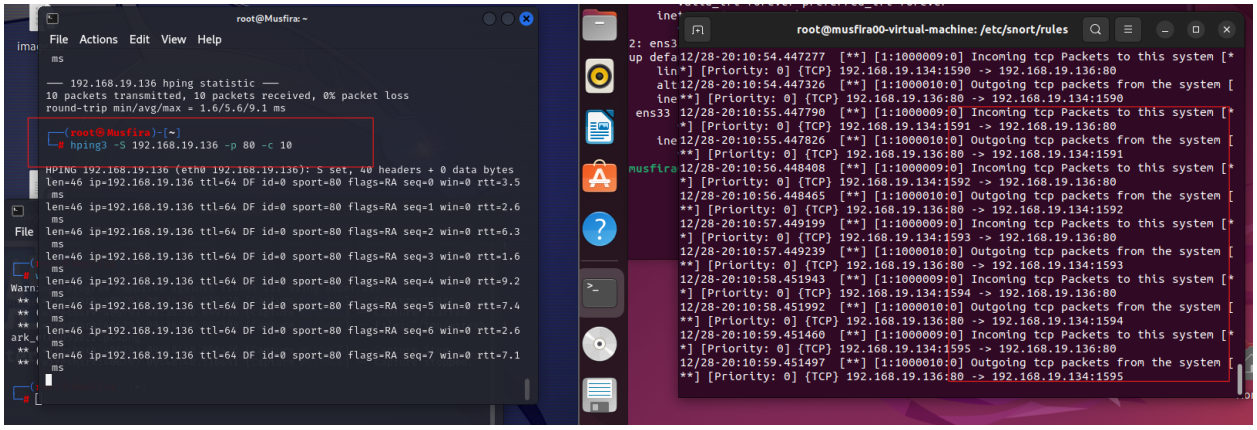
Second Rule — Outgoing TCP from the system: alert tcp 192.168.19.136 any -> any any (msg:"Outgoing tcp packets from the system"; sid:1000010;)

Figure: Outgoing TCP capture/alert rule.

```
alert tcp 192.168.19.136 any -> any any (msg: "Outgoing tcp packets from the system"; sid 1000010;)
```

Snort capturing outgoing and incoming TCP packets:

Figure (Evidence): Snort output showing captured incoming and outgoing TCP packets for system IP 192.168.19.136.



8. How Snort Was Configured & Run

- Install Snort on Ubuntu and place custom rules in the local rules file (e.g., /etc/snort/rules/local.rules)
- Include local.rules in snort.conf
- Start Snort in IDS mode on the monitoring interface, for example:

sudo snort -A console -q -c /etc/snort/snort.conf -i eth0

- From the attacker machine, run the ICMP/TCP/UDP/HTTP tests listed above
- Observe console alerts and review logs under /var/log/snort/ (or configured log path)

9. Response Mechanism

When a rule matched, Snort generated real-time console alerts and logged events. This acts as the detection/response trigger for the IDS: security analysts can investigate the source IP, protocol, and attack pattern from the alert message and packet logs. Optional enhancement: forward alerts to a SIEM/dashboard for visualization.

10. Results & Observations

- Snort successfully detected ICMP, TCP, HTTP, and UDP attack/test traffic.
- Website-visit rules triggered when monitored HTTP/HTTPS destinations were accessed.
- UDP/DNS abnormal traffic generated by hping3 produced alerts as expected.
- Incoming and outgoing TCP traffic to/from the system IP was captured and logged.
- Alerts confirmed that custom rules were loaded correctly and the IDS was actively monitoring.

11. Conclusion

This task demonstrates a practical Network Intrusion Detection System using Snort. By writing custom rules for multiple protocols and simulating realistic attack traffic, the IDS was able to detect suspicious activity, raise alerts, and provide packet-level evidence for further analysis. The exercise covers CodeAlpha Task 4 requirements: IDS setup, rule configuration, continuous monitoring, and alert-based response.

— End of Report —